Object-Oriented Programming in Java

FOUNDATION

The Class: Core of Java

The class is the logical construct upon which the entire Java language is built. It defines the shape and nature of an object, forming the basis for object-oriented programming in Java. Any concept you wish to implement must be encapsulated within a class.

A class creates a new data type that defines both the nature of data being manipulated and the routines used to manipulate it. Think of it as a template or blueprint from which objects are created.

Understanding Classes and Objects

Class: The Template

A class is a **template** for an object. It defines the structure and behavior but doesn't occupy memory until instantiated.

- Logical construct
- Defines data and methods
- Blueprint for objects

Object: The Instance

An object is an **instance** of a class. It has physical reality and occupies space in memory.

- Physical reality
- Contains actual data
- Created using 'new'

General Form of a Class

A class declaration specifies its exact form and nature by defining the data it contains and the code that operates on that data.

```
class classname {
    type instance-variable1;
    type instance-variable2;
    // ...
    type instance-variableN;

    type methodname1(parameter-list) {
        // body of method
    }
    type methodname2(parameter-list) {
        // body of method
    }
    // ...
}
```

The data defined within a class are called **instance variables**. The code is contained within **methods**. Collectively, methods and variables are called **members** of the class.

A Simple Class Example: Box

Let's examine a simple class that defines a box with three dimensions:

```
class Box {
    double width;
    double height;
    double depth;
}
```

This class defines a new data type called Box. However, this declaration only creates a template—it doesn't create an actual object. To create a Box object, you use: Box mybox = new Box();

Instance Variables

Unique Copies

Each object contains its own copy of instance variables

Separate Data

Data for one object is independent from another

Encapsulation

Variables are typically accessed through methods

Variables defined within a class are called instance variables because each instance of the class contains its own copy. Changes to one object's instance variables have no effect on another object's variables.

The Dot Operator

To access instance variables and methods within an object, you use the **dot operator (.)**, which links the object name with the member name.

```
Box mybox = new Box();

// Assign values using dot operator
mybox.width = 10;
mybox.height = 20;
mybox.depth = 15;

// Access values
double w = mybox.width;
```

Syntax

`object.member`

The dot operator provides access to both instance variables and methods of an object.

Box Class Usage

```
class Box {
    double width;
    double height;
    double depth;
}

// This class declares an object of type Box.
class BoxDemo {
    public static void main(String args[]) {
        Box mybox = new Box();
        double vol;

        // assign values to mybox's instance variables
        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;

        // compute volume of box
        vol = mybox.width * mybox.height * mybox.depth;

        System.out.println("Volume is " + vol);
    }
}
```

Declaring Objects: Two-Step Process

1

Step 1: Declare Reference

```
Box mybox;
```

Creates a variable that can refer to an object, but doesn't create the object itself

2

Step 2: Allocate Object

```
mybox = new Box();
```

Dynamically allocates memory for the object and returns a reference to it

These two steps can be combined: Box mybox = new Box();

The 'new' Operator

The **new** operator dynamically allocates memory for an object at runtime and returns a reference to it.

Dynamic Allocation

Memory is allocated during program execution, not at compile time

Flexibility

Programs can create as many or as few objects as needed

Constructor Call

The parentheses after the class name call the constructor

General form: `class-var = new classname();`

Java's primitive types (int, double, etc.) are not implemented as objects for efficiency reasons, so they don't require the new operator.

Object Reference Variables

Understanding how object references work is crucial. When you assign one object reference to another, both refer to the **same object**.

```
Box b1 = new Box();
Box b2 = b1;  // b2 now refers to the same object as b1
```

Important: Assignment copies the reference, not the object. Changes made through b2 will affect the object that b1 refers to, since they're the same object.

Multiple Objects

```
class BoxDemo2 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        // assign values to mybox1's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;

        /* assign different values to mybox2's
        instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;
```

Multiple Objects

```
        // compute volume of first box
        vol = mybox1.width * mybox1.height * mybox1.depth;
        System.out.println("Volume is " + vol);

        // compute volume of second box
        vol = mybox2.width * mybox2.height * mybox2.depth;
        System.out.println("Volume is " + vol);
    }
}
```

METHODS

Introducing Methods

Methods define the behavior of a class. They contain code that operates on the instance variables and determine how the class data can be used.

General Form

```
type name(parameter-list) {
    // body of method
}
```

Key Points

- Type specifies return value
- Use 'void' for no return
- Parameters are optional

Methods provide a controlled interface to the class, allowing you to hide implementation details while exposing functionality.

Adding a Method to Box

Let's add a method to calculate and display the volume of a box:

```
class Box {
    double width;
    double height;
    double depth;

    // display volume of a box
    void volume() {
        System.out.print("Volume is ");
        System.out.println(width * height * depth);
    }
}
```

The method can directly access instance variables without using the dot operator because it's part of the same class.

Calling Methods

Methods are invoked using the dot operator on an object:

```
Box mybox1 = new Box();
mybox1.width = 10;
mybox1.height = 20;
mybox1.depth = 15;

mybox1.volume();  // Calls volume() on mybox1
```

Method Invocation

Control transfers to the method code when called

Execution

Statements inside the method execute

Return

Control returns to the calling code

Instance Method Usage

```
class BoxDemo3 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();

        // assign values to myboxl's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;

        /* assign different values to mybox2's
        instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;

        mybox1.volume();
        mybox2.volume();
    }
}
```

Returning Values from Methods

Methods can return values to the caller. Here's an improved version that returns the volume:

```
double volume() {
    return width * height * depth;
}
```

Usage

```
double vol;
vol = mybox1.volume();
System.out.println("Volume is " + vol);
```

Requirements

- Return type must match declaration
- Receiving variable must be compatible
- Use 'return' statement

Return Usage

```
// Now, volume() returns the volume of a box.
class Box {
    double width;
    double height;
    double depth;

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}
```

Return Usage

```
class BoxDemo4 {
   public static void main(String args[]) {
      Box mybox1 = new Box();
      Box mybox2 = new Box();
      double vol;

      // assign values to myboxl's instance variables
      mybox1.width = 10;
      mybox1.height = 20;
      mybox1.depth = 15;

      // assign different values to mybox2’s instance variables
      mybox2.width = 3;
      mybox2.height = 6;
      mybox2.depth = 9;

      vol = mybox1.volume();
      System.out.println("Volume is " + vol);

      vol = mybox2.volume();
      System.out.println("Volume is " + vol);
   }
}
```

Methods with Parameters

Parameters allow methods to be generalized and operate on different data. A **parameter** is a variable that receives a value when the method is called. An **argument** is the value passed to the method.

```
int square(int i) {
    return i * i;
}

// Usage
int x = square(5);  // x equals 25
int y = square(9);  // y equals 81
```

The parameter 'i' receives the argument value when square() is called.

Parameterized Method Example

Adding a setDim() method to Box that takes dimensions as parameters:

```
void setDim(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
}

// Usage
Box mybox = new Box();
mybox.setDim(10, 20, 15);
```

1

Values Passed

10, 20, 15 are copied into w, h, d

2

Assignment

Parameters assigned to instance variables

3

Result

Object dimensions are set

Parameterized Method Usage

```
// This program uses a parameterized method.
class Box {
    double width;
    double height;
    double depth;

    // compute and return volume
    double volume() {
        return width * height * depth;
    }

    // sets dimensions of box
    void setDim(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }
}
```

Parameterized Method Usage

```
class BoxDemo5 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        // initialize each box
        mybox1.setDim(10, 20, 15);
        mybox2.setDim(3, 6, 9);

        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);

        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume is " + vol);
    }
}
```

INPUT/OUTPUT

Basic Input and Output

Java provides several ways to perform input and output operations. For basic console I/O, we use System.out for output and Scanner for input.

System.out.println()

Prints text followed by a newline

System.out.print()

Prints text without newline

Scanner Class

Reads input from various sources

Buffered Reader Class

Reads input from character i/p stream

Console Class

Reads input from console devices

Scanner Class

The **Scanner** class is a versatile utility used to parse and break input into tokens using a delimiter. A **token** is a small element of data (like a word or number), and a **delimiter** is the character (default is whitespace) used to separate these tokens.

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter an integer: ");
int num = sc.nextInt(); // Directly parses input into a primitive type
```

- Provides convenient methods like nextInt() and nextDouble() to parse primitive data types directly.
- Can read input from keyboards, files, or strings using customizable delimiters (default is whitespace).
- Ideal for beginners and simple interactive applications due to its straightforward syntax.

Example of Scanner Class

```
import java.util.Scanner;

public class ScannerExample {
    public static void main(String[] args) {
        // Create scanner object to read from standard input
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine(); // Reads the whole line

        System.out.print("Enter your age: ");
        int age = sc.nextInt(); // Automatically parses text to integer

        System.out.println("Hello " + name + "! You are " + age + " years old.");

        sc.close(); // Important to close resources
    }
}
```

Buffered Reader Class

The **BufferedReader** class is a character-stream reader that uses an internal buffer to store data for high-performance reading. A **buffer** is a temporary storage area in memory that reduces the number of slow I/O operations by reading large "chunks" of data at once.

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
String line = br.readLine(); // Efficiently reads an entire line of text
```

- Significantly faster than Scanner for large datasets because it minimizes disk/network access.
- Excellent for processing text line-by-line using the readLine() method.
- It is synchronized, making it the preferred choice for multi-threaded applications.

Example of BufferedReader Class

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class BufferedReaderExample {
    public static void main(String[] args) throws IOException {
        // Wrap System.in in an InputStreamReader, then in a BufferedReader
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));

        System.out.print("Enter a line of text: ");
        String input = reader.readLine(); // Highly efficient for line reading

        System.out.println("You typed: " + input);
        // Note: Closing standard input is not strictly required but reader.close() works
    }
}
```

Console Class

The **Console** class provides specialized methods to interact with the system's character-based terminal device. It is unique because it can read **secure input** (like passwords) without displaying the characters on the screen, known as "echoing".

```
Console con = System.console();
if (con != null) {
    char[] pass = con.readPassword("Enter key: "); // Secure, no-echo input
}
```

- The readPassword() method returns a char[] instead of a String to prevent sensitive data from lingering in memory.
- Only works in a physical terminal/command prompt; it usually returns null in most IDEs like Eclipse or IntelliJ.
- Includes a printf() method for convenient, formatted output directly to the console.

Example of Console Class

```
import java.io.Console;

public class ConsoleExample {
    public static void main(String[] args) {
        // Get unique console object from the system
        Console con = System.console();

        if (con == null) {
            System.out.println("No console available (Running in IDE?)");
            return;
        }

        String username = con.readLine("Username: ");
        // readPassword returns char[] for better security than String
        char[] password = con.readPassword("Password: ");

        con.printf("User %s is now logged in.%n", username);
    }
}
```

CONSTRUCTORS

Introducing Constructors

A **constructor** initializes an object immediately upon creation. It has the same name as the class and is automatically called when an object is created.

Same Name

Constructor name must match the class name exactly

No Return Type

Constructors have no return type, not even void

Automatic Call

Called automatically when 'new' creates an object

Simple Constructor Example

Here's a Box class with a constructor that initializes dimensions:

```
class Box {
    double width;
    double height;
    double depth;

    // This is the constructor for Box
    Box() {
        System.out.println("Constructing Box");
        width = 10;
        height = 10;
        depth = 10;
    }

    double volume() {
        return width * height * depth;
    }
}
```

When you create a Box object with `Box mybox = new Box();`, the constructor is automatically called, initializing all dimensions to 10.

Parameterized Constructors

Constructors can take parameters to initialize objects with specific values:

```
Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
}

// Usage
Box mybox1 = new Box(10, 20, 15);
Box mybox2 = new Box(3, 6, 9);
```

Each object is initialized with the values specified in its constructor call. This is much more useful than having all boxes with the same dimensions.

The 'this' Keyword

The **this** keyword refers to the current object—the object on which the method was invoked.

Basic Usage

```
Box(double w, double h, double d) {
    this.width = w;
    this.height = h;
    this.depth = d;
}
```

When to Use

- Resolve name conflicts
- Pass current object to methods
- Call other constructors

Using 'this' is particularly useful when parameter names match instance variable names.

Constructor Usage

```
class Box {
    double width;
    double height;
    double depth;

    // This is the constructor for Box.
    Box(double width, double height, double depth) {
        this.width = width;
        this.height = height;
        this.depth = depth;
    }

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}
```

Constructor Usage

```
class BoxDemo7 {
    public static void main(String args[]) {
        // declare, allocate, and initialize Box objects
        Box mybox1 = new Box(10, 20, 15);
        Box mybox2 = new Box(3, 6, 9);
        double vol;

        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);

        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume is " + vol);
    }
}
```

Instance Variable Hiding

When a parameter has the same name as an instance variable, the parameter **hides** the instance variable. Use 'this' to access the instance variable:

```
Box(double width, double height, double depth) {
    this.width = width;    // this.width is instance variable
    this.height = height;  // width is parameter
    this.depth = depth;
}
```

While you can use different names to avoid hiding, many programmers prefer using the same names with 'this' for clarity.

OVERLOADING

Method Overloading

Java allows you to define multiple methods with the same name, as long as their parameter lists differ. This is called **method overloading**.

Same Name

Methods share the same name

Different Parameters

Must differ in number or type of parameters

Compile-Time Resolution

Java determines which version to call based on arguments

Method Overloading Example

```
class OverloadDemo {
    void test() {
        System.out.println("No parameters");
    }

    void test(int a) {
        System.out.println("a: " + a);
    }

    void test(int a, int b) {
        System.out.println("a and b: " + a + " " + b);
    }

    double test(double a) {
        System.out.println("double a: " + a);
        return a * a;
    }
}
```

Java uses the type and number of arguments to determine which version to call. Return types alone cannot distinguish overloaded methods.

Method Overloading Usage

```
class Overload {
    public static void main(String args[]) {
        OverloadDemo ob = new OverloadDemo();
        double result;

        // call all versions of test()
        ob.test();
        ob.test(10);
        ob.test(10, 20);
        result = ob.test(123.25);
        System.out.println("Result of ob.test(123.25): " + result);
    }
}
```

Automatic Type Conversion in Overloading

When no exact match is found, Java can use automatic type conversions to resolve method calls:

```
void test(double a) {
    System.out.println("double a: " + a);
}

// Usage
int i = 88;
test(i);  // Converts int to double
```

Conversion Rules

- Only if no exact match exists
- Follows standard type promotion
- int → double is allowed

Overloading Constructors

Constructors can also be overloaded, allowing objects to be initialized in different ways:

```
class Box {
    double width, height, depth;

    // Constructor for all dimensions
    Box(double w, double h, double d) {
        width = w; height = h; depth = d;
    }

    // Constructor for no dimensions
    Box() {
        width = height = depth = -1;
    }

    // Constructor for cube
    Box(double len) {
        width = height = depth = len;
    }
}
```

This provides flexibility in object creation: `Box b1 = new Box(10,20,15);`, `Box b2 = new Box();`, Box b3 = new Box(7);

Constructor Overloading Usage

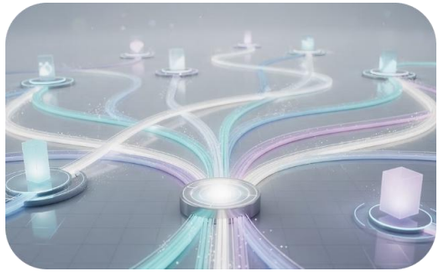
```
class OverloadCons {
    public static void main(String args[]) {
        // create boxes using the various constructors
        Box mybox1 = new Box(10, 20, 15);
        Box mybox2 = new Box();
        Box mycube = new Box(7);
        double vol;

        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " + vol);

        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume of mybox2 is " + vol);

        // get volume of cube
        vol = mycube.volume();
        System.out.println("Volume of mycube is " + vol);
    }
}
```

Benefits of Constructor Overloading



Flexibility

Objects can be created in multiple convenient ways

Code Clarity

Enhances readability and maintainability of code

Convenience

Users can choose the most appropriate constructor for their needs

PARAMETERS

Using Objects as Parameters

Methods can accept objects as parameters, just like primitive types. This is common and useful for comparing objects or initializing one object from another.

```
class Test {
 int a, b;

 Test(int i, int j) {
 a = i; b = j;
 }

 // Method that takes an object as parameter
 boolean equalTo(Test o) {
 if(o.a == a && o.b == b)
 return true;
 else
 return false;
 }
}
```

Objects as Parameters Usage

```
class PassOb {
    public static void main(String args[]) {
        Test ob1 = new Test(100, 22);
        Test ob2 = new Test(100, 22);
        Test ob3 = new Test(-1, -1);

        System.out.println("ob1 == ob2: " + ob1.equalTo(ob2));
        System.out.println("ob1 == ob3: " + ob1.equalTo(ob3));
    }
}
```

Object Parameters in Constructors

A common use of object parameters is in constructors, allowing one object to initialize another:

```
class Box {
    double width, height, depth;

    // Constructor that takes an object
    Box(Box ob) {
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }

    // Other constructors...
}

// Usage
Box mybox1 = new Box(10, 20, 15);
Box myclone = new Box(mybox1);  // Create copy
```

This creates a new object with the same dimensions as an existing object.

Objects as Parameters Usage

```
class Box {
    double width;
    double height;
    double depth;

    // Notice this constructor. It takes an object of type Box.
    Box(Box ob) { // pass object to constructor
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }
     Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }
     Box() {
        width = -1; // use -1 to indicate
        height = -1; // an uninitialized
        depth = -1; // box
    }
```

Objects as Parameters Usage

```
    // constructor used when cube is created
    Box(double len) {
        width = height = depth = len;
    }

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}
```

Objects as Parameters Usage

```
class OverloadCons2 {
    public static void main(String args[]) {
        // create boxes using the various constructors
        Box mybox1 = new Box(10, 20, 15);
        Box mybox2 = new Box();
        Box mycube = new Box(7);
        Box myclone = new Box(mybox1); // create copy of mybox1
        double vol;

        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " + vol);

        vol = mybox2.volume();
        System.out.println("Volume of mybox2 is " + vol);

        vol = mycube.volume();
        System.out.println("Volume of cube is " + vol);

        vol = myclone.volume();
        System.out.println("Volume of clone is " + vol);
    }
}
```

Argument Passing: Call-by-Value

Java uses **call-by-value** for all arguments. However, the effect differs between primitive types and reference types.

Primitive Types

A copy of the value is passed. Changes to the parameter don't affect the original argument.

```
void meth(int i) {
i *= 2;  // Only affects local copy
}
```

Reference Types

A copy of the reference is passed. Changes to the object affect the original because both references point to the same object.

```
void meth(Test o) {
 o.a *= 2; // Affects original object
}
```

Primitive Types: Pass-by-Value

When primitive types are passed to methods, changes inside the method don't affect the original variables:

```
class Test {
    void meth(int i, int j) {
        i *= 2;
        j /= 2;
    }
}

// Usage
int a = 15, b = 20;
ob.meth(a, b);
// a is still 15, b is still 20
```

The operations inside meth() have no effect on a and b because only copies were passed.

Pass-by-Value Usage

```
class Test {
    void meth(int i, int j) {
        i *= 2;
        j /= 2;
    }
}

class CallByValue {
    public static void main(String args[]) {
        Test ob = new Test();
        int a = 15, b = 20;

        System.out.println("a and b before call: " + a + " " + b);

        ob.meth(a, b);

        System.out.println("a and b after call: " + a + " " + b);
    }
}
```

Objects: Effective Pass-by-Reference

When objects are passed, the reference is copied, but both references point to the same object:

```
class Test {
    int a, b;

    void meth(Test o) {
        o.a *= 2;
        o.b /= 2;
    }
}

// Usage
Test ob = new Test(15, 20);
ob.meth(ob);
// ob.a is now 30, ob.b is now 10
```

Changes made through the parameter affect the original object because they refer to the same object in memory.

Pass-by-Reference Usage

```
class Test {
    int a, b;

    Test(int i, int j) {
        a = i;
        b = j;
    }

    // pass an object
    void meth(Test o) {
        o.a *= 2;
        o.b /= 2;
    }
}
```

Pass-by-Reference Usage

```
class PassObjRef {
    public static void main(String args[]) {
        Test ob = new Test(15, 20);

        System.out.println("ob.a and ob.b before call: " +
                           ob.a + " " + ob.b);

        ob.meth(ob);

        System.out.println("ob.a and ob.b after call: " +
                           ob.a + " " + ob.b);
    }
}
```

Returning Objects

Methods can return objects, just like any other type. The returned object continues to exist as long as there's a reference to it.

```
class Test {
    int a;

    Test(int i) {
        a = i;
    }

    Test incrByTen() {
        Test temp = new Test(a + 10);
        return temp;
    }
}

// Usage
Test ob1 = new Test(2);
Test ob2 = ob1.incrByTen();  // ob2.a is 12
```

Each call creates a new object and returns a reference to it.

Returning Object Usage

```
class Test {
    int a;
    Test(int i) {
        a = i;
    }
    Test incrByTen() {
        Test temp = new Test(a + 10);
        return temp;
    }
}
class RetOb {
    public static void main(String args[]) {
        Test ob1 = new Test(2);
        Test ob2;
        ob2 = ob1.incrByTen();
        System.out.println("ob1.a: " + ob1.a);
        System.out.println("ob2.a: " + ob2.a);
        ob2 = ob2.incrByTen();
        System.out.println("ob2.a after second increase: " + ob2.a);
    }
}
```

RECURSION

Understanding Recursion

Recursion is the process of defining something in terms of itself. A recursive method is one that calls itself.

Method Called

Initial invocation with parameter

Base Case Check

Is termination condition met?

Return

Results combine as calls return

Recursive Call

Method calls itself with modified parameter

Factorial: Classic Recursion Example

Computing factorial demonstrates recursion perfectly. Factorial of n is n × (n-1) × (n-2) × ... × 1.

```
class Factorial {
    int fact(int n) {
        int result;

        if(n == 1)
            return 1;
        result = fact(n-1) * n;
        return result;
    }
}

// fact(5) returns 120
// fact(4) returns 24
// fact(3) returns 6
```

When fact(3) is called, it calls fact(2), which calls fact(1). Then the calls return: fact(1)→1, fact(2)→2, fact(3)→6.

Recursion Usage

```
class Recursion {
    public static void main(String args[]) {
        Factorial f = new Factorial();
        System.out.println("Factorial of 3 is " + f.fact(3));
        System.out.println("Factorial of 4 is " + f.fact(4));
        System.out.println("Factorial of 5 is " + f.fact(5));
    }
}
```

How Recursion Works

fact(3) called

Calls fact(2) and waits

fact(2) called

Calls fact(1) and waits

fact(1) called

Returns 1 (base case)

fact(2) returns

Returns 1 × 2 = 2

fact(3) returns

Returns 2 × 3 = 6

New local variables and parameters are allocated on the stack for each call. As calls return, old variables are removed from the stack.

Recursion Considerations

Advantages

- Clearer, simpler code for some algorithms
- Natural fit for tree/graph problems
- Elegant mathematical solutions

Disadvantages

- Slightly slower than iteration
- Risk of stack overflow
- Must have base case to terminate

Critical: Always include a base case (termination condition) or the method will recurse infinitely and cause a stack overflow.

ACCESS CONTROL

Introducing Access Control

Access control allows you to prevent misuse of class members by controlling what parts of a program can access them. This is a key aspect of encapsulation.

public

Accessible by any other code

private

Accessible only within its own class

protected

Accessible within package and subclasses

default

Accessible within the same package

Public and Private Access

The two most commonly used access modifiers are **public** and **private**.

```
class Test {
    int a;              // default access
    public int b;       // public access
    private int c;      // private access

    // Methods to access c
    void setc(int i) {
        c = i;
    }

    int getc() {
        return c;
    }
}
```

Variable 'c' cannot be accessed directly from outside the class. It must be accessed through setc() and getc() methods.

Access Control Usage

```
class AccessTest {
    public static void main(String args[]) {
        Test ob = new Test();

        // These are OK, a and b may be accessed directly
        ob.a = 10;
        ob.b = 20;

        // This is not OK and will cause an error
        // ob.c = 100; // Error!

        // You must access c through its methods
        ob.setc(100); // OK
        System.out.println("a, b, and c: " + ob.a + " " + ob.b + " " + ob.getc());
    }
}
```

Why Use Private Access?

Data Protection

Prevents accidental or malicious alteration of critical data

Implementation Hiding

Internal details can change without affecting external code

Controlled Interface

Access only through well-defined methods ensures data integrity

Validation

Methods can validate data before allowing changes

Improved Stack with Access Control

Here's the Stack class with proper access control:

```
class Stack {
    private int stk[] = new int[10];
    private int tos;

    Stack() {
        tos = -1;
    }

    void push(int item) {
        if(tos == 9)
            System.out.println("Stack is full.");
        else
            stk[++tos] = item;
    }

    int pop() {
        if(tos < 0) {
            System.out.println("Stack underflow.");
            return 0;
        }
        else
            return stk[tos--];
    }
}
```

Now stk and tos are private, preventing external code from corrupting the stack.

Stack Usage

```
class TestStack {
    public static void main(String args[]) {
        Stack mystack1 = new Stack();
        Stack mystack2 = new Stack();

        // push some numbers onto the stack
        for(int i=0; i<10; i++) mystack1.push(i);
        for(int i=10; i<20; i++) mystack2.push(i);

        // pop those numbers off the stack
        System.out.println("Stack in mystack1:");
        for(int i=0; i<10; i++)
            System.out.println(mystack1.pop());

        System.out.println("Stack in mystack2:");
        for(int i=0; i<10; i++)
            System.out.println(mystack2.pop());

        // these statements are not legal
        // mystack1.tos = -2;
        // mystack2.stck [3] = 100;
    }
}
```

STATIC

Understanding Static Members

The **static** keyword creates members that can be used independently of any object. Static members belong to the class itself, not to instances.

Class-Level

Belongs to the class, not individual objects

Shared

All instances share the same static variable

No Object Needed

Can be accessed before any objects exist

Static Variables

Static variables are essentially global variables. When objects are created, no copy of a static variable is made—all instances share it.

```
class StaticDemo {
    static int a = 42;
    static int b = 99;

    static void callme() {
        System.out.println("a = " + a);
    }
}

// Access without creating object
StaticDemo.callme();
System.out.println("b = " + StaticDemo.b);
```

Static members are accessed using the class name followed by the dot operator: ClassName.member

Static Usage

```
class StaticByName {
    public static void main(String args[]) {
        StaticDemo.callme();
        System.out.println("b = " + StaticDemo.b);
    }
}
```

Static Methods

Static methods have several restrictions because they exist independently of any object:

1

Static Methods Only

Can only directly call other static methods of their class

2

Static Variables Only

Can only directly access static variables of their class

3

No 'this' or 'super'

Cannot refer to 'this' or 'super' in any way

The main() method is static because it must be called before any objects exist.

Static Initialization Blocks

Static blocks execute once when the class is first loaded, before any objects are created:

```
class UseStatic {
    static int a = 3;
    static int b;

    static {
        System.out.println("Static block initialized.");
        b = a * 4;
    }

    public static void main(String args[]) {
        System.out.println("a = " + a);
        System.out.println("b = " + b);
    }
}
```

Output: Static block initialized. / a = 3 / b = 12

FINAL

Introducing Final

The **final** keyword prevents modification. When applied to variables, it creates constants.

Final Variables

```
final int FILE_NEW = 1;
final int FILE_OPEN = 2;
final int FILE_SAVE = 3;
final int FILE_QUIT = 5;
```

Characteristics

- Must be initialized when declared
- Cannot be changed after initialization
- Convention: use UPPERCASE names

Final can also be applied to method parameters and local variables to prevent reassignment.

Final Variables in Practice

Constants

Define unchangeable values like mathematical constants or configuration values

Parameters

Prevent method parameters from being modified within the method

Local Variables

Ensure a local variable is assigned only once

```
void process(final int value) {
 // value cannot be changed
 final int result = value * 2;
 // result cannot be changed
}
```

ARRAYS

Arrays Revisited

Arrays in Java are implemented as objects. This means they have special properties, most notably the **length** instance variable.

```
int a1[] = new int[10];
int a2[] = {3, 5, 7, 1, 8, 99, 44, -10};

System.out.println("length of a1 is " + a1.length);  // 10
System.out.println("length of a2 is " + a2.length);  // 8
```

The length member always holds the number of elements the array can hold, not the number currently in use.

Using Array Length

The length member is useful for preventing array overflows and creating flexible code:

```
class Stack {
    private int stk[];
    private int tos;

    Stack(int size) {
        stk = new int[size];
        tos = -1;
    }

    void push(int item) {
        if(tos == stk.length-1)  // Use length
            System.out.println("Stack is full.");
        else
            stk[++tos] = item;
    }
    // ...
}
```

This allows creating stacks of any size: `Stack s1 = new Stack(5);` or Stack s2 = new Stack(100);

Stack Usage

```
class Stack {
    private int stck[];
    private int tos;

    Stack(int size) {
        stck = new int[size];
        tos = -1;
    }

    void push(int item) {
        if (tos == stck.length - 1) // use length member
            System.out.println("Stack is full.");
        else
            stck[++tos] = item;
    }

    int pop() {
        if (tos < 0) {
            System.out.println("Stack underflow.");
            return 0;
        } else {
            return stck[tos--];
        }
    }
}
```

Stack Usage

```
class TestStack2 {
    public static void main(String args[]) {
        Stack mystack1 = new Stack(5);
        Stack mystack2 = new Stack(8);

        // push some numbers onto the stack
        for (int i = 0; i < 5; i++)
            mystack1.push(i);
        for (int i = 0; i < 8; i++)
            mystack2.push(i);

        // pop those numbers off the stack
        System.out.println("Stack in mystack1:");
        for (int i = 0; i < 5; i++)
            System.out.println(mystack1.pop());

        System.out.println("Stack in mystack2:");
        for (int i = 0; i < 8; i++)
            System.out.println(mystack2.pop());
    }
}
```

NESTED CLASSES

Nested and Inner Classes

A **nested class** is a class defined within another class. An **inner class** is a non-static nested class.

Scope

Bounded by enclosing class

Access

Can access all members of outer class

Organization

Improves code organization

Encapsulation

Logically groups related classes

Inner Class Example

```
class Outer {
    int outer_x = 100;
    void test() {
        Inner inner = new Inner();
        inner.display();
    }
    // This is an inner class
    class Inner {
        void display() {
            System.out.println("display: outer_x = " + outer_x);
        }
    }
}
class InnerClassDemo {
    public static void main(String args[]) {
        Outer outer = new Outer();
        outer.test();
    }
}
```

The Inner class can directly access outer_x because it's defined within Outer's scope.

Inner Class Characteristics

Access Rights

- Inner class accesses all outer class members
- Outer class cannot access inner class members
- Inner class instance requires outer class instance

Use Cases

- Event handlers
- Iterator implementations
- Helper classes
- Callbacks

Inner classes can even be defined within method blocks or loop bodies for very localized functionality.

STRINGS

The String Class

Strings are one of the most commonly used classes in Java. Every string you create is actually a String object.

Immutable

Once created, String contents cannot be altered

Objects

Even string literals are String objects

Concatenation

Use + operator to combine strings

```
String myString = "this is a test";
System.out.println(myString);

String combined = "I" + " like " + "Java.";
```

String Methods

The String class provides many useful methods for working with strings:

```
String str1 = "First String";
String str2 = "Second String";

// Get length
int len = str1.length();  // 12

// Get character at index
char ch = str1.charAt(3);  // 's'

// Compare strings
if(str1.equals(str2))
    System.out.println("Equal");
else
    System.out.println("Not equal");
```

Important: Use equals() to compare string contents, not == which compares references.

Stack Usage

```
class StringDemo2 {
    public static void main(String args[]) {
        String strOb1 = "First String";
        String strOb2 = "Second String";
        String strOb3 = strOb1;

        System.out.println("Length of strOb1: " +
                           strOb1.length());

        System.out.println("Char at index 3 in strOb1: " +
                           strOb1.charAt(3));

        if (strOb1.equals(strOb2))
            System.out.println("strOb1 == strOb2");
        else
            System.out.println("strOb1 != strOb2");

        if (strOb1.equals(strOb3))
            System.out.println("strOb1 == strOb3");
        else
            System.out.println("strOb1 != strOb3");
    }
}
```

String Arrays

You can create arrays of strings just like arrays of any other type:

```
String str[] = { "one", "two", "three" };

for(int i=0; i < str.length; i++)
    System.out.println("str[" + i + "]: " + str[i]);

// Output:
// str[0]: one
// str[1]: two
// str[2]: three
```

Storage

Each element holds a reference to a String object

Common Use

Command-line arguments are passed as String arrays

COMMAND LINE

Command-Line Arguments

Command-line arguments allow you to pass information to a program when it runs. They're stored in the String array passed to main().

```
class CommandLine {
 public static void main(String args[]) {
 for(int i=0; i < args.length; i++)
 System.out.println("args[" + i + "]: " + args[i]);
 }
}

// Execute: java CommandLine this is a test 100
// Output:
// args[0]: this
// args[1]: is
// args[2]: a
// args[3]: test
// args[4]: 100
```

Working with Command-Line Arguments

1. **Passed as Strings**
   All arguments are String type
2. **Stored in Array**
   args[0] is first argument
3. **Check Length**
   Use args.length to count
4. **Convert Types**
   Parse numeric values manually

Remember: All command-line arguments are passed as strings. You must convert numeric values to their internal forms using methods like Integer.parseInt().

Key Concepts Summary

1

Classes & Objects

Classes are templates; objects are instances with physical reality

2

Methods

Define behavior and provide controlled access to data

3

Constructors

Initialize objects automatically upon creation

4

Access Control

Public and private modifiers protect data integrity

5

Static Members

Belong to class, not instances; shared by all objects

6

Overloading

Multiple methods with same name but different parameters

Mastering Object-Oriented Programming

You've now explored the fundamental concepts of object-oriented programming in Java. These building blocks—classes, objects, methods, constructors, and access control—form the foundation for creating robust, maintainable software.

Practice

Apply these concepts in your own programs to solidify understanding

Experiment

Try different combinations of features to see how they interact

Build

Create increasingly complex classes that solve real problems

The journey to Java mastery continues with inheritance, polymorphism, and advanced OOP concepts in the next chapters.